

Task 3 — The Agent Loop: Planning, Tool Use, and MCP

TRACK Agentic AI — Intern Programme, Task 3 of 5

PACKAGE	packages/agent
PREREQUISITES	TypeScript, completion of Task 2 (or equivalent retrieval/tool-calling familiarity).
COST	Zero. Everything runs locally.

Why this task exists

Everyone can call an LLM once and print the answer. The entire discipline of “agentic AI” is what happens when the model is wrong the first time — does the system notice, and does it recover, within a budget, without doing something destructive. A RAG pipeline retrieves and answers once. An agent decides, acts, observes the result, and decides again. That loop, and the guardrails around it, is what you are building.

Most people who build “an agent” wire a while-loop around some tool calls and call it done. It works on the happy path and never stops on the unhappy one — it burns budget retrying the same failed action, or it writes outside the directory it was told to touch. You will build the loop, and then spend at least as much effort making it stop safely as making it work at all.

Setup

Runtime

COMPONENT	CHOICE	WHY
Node	20 or newer	Native fetch
TypeScript	5.x, strict: true	Non-negotiable
Test runner	Vitest	Fast, ESM-native
Model runtime	Ollama, qwen2.5:7b-instruct	Needs real tool-calling, not just chat

If 7b's tool-calling is unreliable, that unreliability is data — record it in NOTES.md rather than quietly swapping models. Tool-call error rate is one of the metrics you're reporting.

Corpus

A small TypeScript repo with a known-broken test suite: 5–10 independent failing tests of varying difficulty — an off-by-one, a wrong import, a mis-typed function signature, a missing null check, a logic error that needs re-reading two files to find. Hand-seed these into a real small repo rather than a synthetic toy; the bugs should look like the kind a tired developer actually leaves behind.

Explicitly forbidden

LangGraph, CrewAI, AutoGen, or any pre-built agent framework. You may read their source. You may not import them. Also forbidden: giving the agent shell access to run arbitrary commands — it gets a fixed, typed tool surface, nothing more.

What you are building

A CLI agent that is handed one failing test at a time, investigates, proposes an edit, applies it, re-runs the test, and repeats until the test passes or its budget runs out.

```
pnpm agent fix --test "auth.redirect.test.ts"
pnpm agent eval                               # runs the golden set, prints metrics
pnpm agent eval --compare baseline.json
```

Requirements

1. The plan-act-observe loop

Each iteration: the model sees the current state (test output, files read so far, prior actions), picks exactly one tool call, the harness executes it, the result is appended to the transcript, and the loop repeats. No multi-tool-call-per-turn free-for-alls — one action, one observation, every time. That discipline is what makes step-by-step evaluation possible at all.

Every run has a step budget (e.g. 12) and a wall-clock budget. Both are enforced by the harness, not requested politely of the model.

2. MCP tool surface

Expose the agent's tools as an actual MCP server (Model Context Protocol) — not just function-calling JSON you invented yourself. This is the same protocol real coding agents expose tools through, and building one from scratch is the fastest way to understand why it looks the way it does: typed schemas, structured results, and a clean boundary between “the model decided” and “the harness executed.”

```
interface ToolCall {
  id: string;
  name: 'read_file' | 'list_dir' | 'grep' | 'propose_edit' | 'run_test';
  arguments: Record<string, unknown>;
}
interface ToolResult {
  id: string;
  ok: boolean;
  output: string;
  truncated?: boolean;
}
```

3. The approval gate

propose_edit never writes directly. It produces a diff. A separate, non-LLM check verifies the diff only touches files inside the repo under test — no path traversal, no touching node_modules, no touching the eval harness itself — before it is applied. This check must fail loudly and abort the run, not silently skip the edit, if it's violated. It's the smallest version of the guardrail any production agent needs before it's allowed near real infrastructure.

4. Halting conditions

- Test passes: success.

- Step budget exhausted: failure, logged with the final state.
- Same tool call with the same arguments repeated 3 times in a row: abort as a stuck loop — don't burn the rest of the budget re-trying it.
- Approval gate violation: abort immediately, log as a distinct safety failure, not a normal failure.

5. Trajectory logging

Every run writes a full JSONL trajectory — one line per step: the model's stated reasoning (if you elicit it), the tool call, the result, and a timestamp. This is your raw material for Task 4 and Task 5, and it's also what makes a failed run debuggable at all instead of just "it didn't work."

6. Golden evaluation set — 15 broken tests

DIFFICULTY	COUNT	DEFINITION
Easy	6	The bug is in the file the failing test imports directly
Medium	6	The fix requires reading 2 files to find the actual cause
Hard	3	The fix touches a shared utility used elsewhere, or the test's own expectation is wrong — include at least one bug outside the tool surface entirely, where the correct answer is to report it can't be fixed, not force an edit

7. Metrics

METRIC	WHAT IT MEASURES
success@budget	Fraction of runs solved within the step budget
mean steps to success	Efficiency, successful runs only
wasted-step ratio	Steps that repeat a prior failed action or re-read a file already in context
tool-call error rate	Malformed calls, wrong arguments, hallucinated file paths
guardrail violations	Should be exactly zero — report it as a distinct metric even so
p50 / p95 latency	Wall-clock per run

Deliverable: RESULTS.md

A table comparing: baseline (model guesses a fix from the failing assertion alone, no exploration) vs + read_file/grep vs + step budget & stuck-loop detection vs + approval gate. Then, in prose, which change moved which metric, and why. If adding grep didn't move success@budget but did drop the wasted-step ratio, say so and explain it.

Hard parts you should expect

- The model re-reading files it already has in context because nothing told it what it's already seen — track “seen” state explicitly in the harness, don't rely on the model remembering its own transcript.
- Diffs that apply cleanly but don't compile — validate with a real typecheck/test run, never trust that a diff looked plausible.
- The stuck-loop case is more common than it sounds: a model that gets a tool error will often retry the identical call rather than adjust it.
- A model that “fixes” the test by weakening the assertion instead of the code — decide in advance whether your approval gate flags edits to test files for higher scrutiny (not a blanket ban, since your own golden set includes cases where the test itself needs care).

Deliverables

- packages/agent — the loop, the MCP tool server, the CLI
- evals/golden-agent.jsonl — 15 hand-verified broken-test scenarios
- Metrics harness reporting all six metrics
- RESULTS.md — the comparison table plus the attribution paragraph
- NOTES.md — why MCP over ad hoc function-calling JSON; what the stuck-loop detector caught
- DESIGN.md — written before the code

Design doc, before any code

One page, reviewed and approved before you write a line:

- Public interfaces and types
- The three most likely failure modes, and your plan for each
- What you are deliberately not building, and why
- Open questions

Review gate

Your reviewer will verify these by hand before merge.

- Kill the approval gate in code and confirm your eval harness catches it — a canary test in your own suite, not just your word for it.
- Ask the agent to fix a test whose real bug is outside its tool surface (e.g. an environment variable). It should report this rather than fabricate a fix.
- Feed it a broken test with an intentionally error-prone tool response three times in a row; confirm the stuck-loop detector fires before the step budget is exhausted.
- Every number in RESULTS.md reproduces from a fresh clone.